

B

C

Bachelorarbeit

Compileranbindung für die
Programmier-Lern-Plattform
CodeTask

S

Bachelorarbeit

Compileranbindung für die Programmier-Lern-Plattform CodeTask

von

Cilas Handloser

zur Erlangung des akademischen Grades

Bachelor of Science

in Angewandter Informatik

an der Hochschule Konstanz Technik, Wirtschaft und Gestaltung

Matrikelnummer: 307610

Abgabedatum: 30. Juni 2026

Erstbetreuer: **Prof. Dr. Marko Boger**

Zweitbetreuer: **Prof. Dr. Pascal Laube**

Eine elektronische Version dieser Bachelorarbeit ist verfügbar unter <https://github.com/Ciilas/htwg-latex-BA-Cilas>.

Kurzfassung

Abstract. . .

Inhaltsverzeichnis

1	Einleitung	1
1.1	Document Structure	1
1.2	Bibliography	2
1.3	Cover and Title Page	3
1.4	Chapters	3
1.5	\section{...}	4
1.5.1	\subsection{...}	4
1.6	Fonts and Colors.	4
2	Grundlagen und Technologien	5
2.1	Online-Judge-Systeme	5
2.1.1	Architektur eines Online-Judges	6
2.1.2	Der Prozessablauf	7
2.1.3	Vergleich bestehender Systeme	8
2.2	Containerisierung mit Docker	10
2.2.1	Das Docker-Prinzip (Images und Container)	10
2.2.2	Isolation und Sicherheit (Sandboxing)	12
2.3	Orchestrierung mit Kubernetes	12
2.3.1	Grundlagen der Container Verwaltung.	12
2.3.2	Hochverfügbarkeit und Self-Healing	12
3	Anforderungen	13
4	Konzeptarchitektur	15
5	Implementierung	17
6	Evaluation	19
7	Fazit	21
	Literatur	23

1

Einleitung

This document is intended to be both an example of the HTWG Konstanz $\text{\LaTeX}$ template for reports and theses, as well as a short introduction to its use. It is not intended to be a general introduction to $\text{\LaTeX}$ itself,¹ and we will assume the reader to be familiar with the basics of creating and compiling documents.

1.1. Document Structure

Since a report, and especially a thesis, might be a substantial document, it is convenient to break it up into smaller pieces. In this template we therefore give every chapter its own file. The chapters (and appendices) are gathered together in `report.tex`, which is the master file describing the overall structure of the document. `report.tex` starts with the line

```
\documentclass{htwg-report}
```

which loads the HTWG Konstanz report template. The template is based on the $\text{\LaTeX}$ book document class and stored in `tudelft-report.cls`. The document class accepts several comma-separated options. The default language is English, but this can be changed to Dutch (e.g., for bachelor theses) by specifying the `dutch` option:

```
\documentclass[german]{htwg-report}
```

¹We recommend <http://en.wikibooks.org/wiki/LaTeX> as a reference and a starting point for new users.

Furthermore, hyperlinks are shown in blue, which is convenient when reader the report on a computer, but can be expensive when printing. They can be turned black with the `print` option. This will also turn the headers black instead of cyan.

If the document becomes large, it is easy to miss warnings about the layout in the $\text{\LaTeX}$ output. In order to locate problem areas, add the `draft` option to the `\documentclass` line. This will display a vertical bar in the margins next to the paragraphs that require attention. Finally, the `nativefonts` option can be used to override the automatic font selection (see below).

This template has the option to automatically generate a cover page with the `\makecover` command. See the next section for a detailed description.

The contents of the report are included between the `\begin{document}` and `\end{document}` commands, and split into three parts by

1. `\frontmatter`, which uses Roman numerals for the page numbers and is used for the title page and the table of contents;
2. `\mainmatter`, which uses Arabic numerals for the page numbers and is the style for the chapters;
3. `\appendix`, which uses letters for the chapter numbers, starting with 'A'.

The title page is defined in a separate file, *e.g.*, `title.tex`, and included verbatim with `\input{title}`.² Additionally, it is possible to include a preface, containing, for example, the acknowledgements. An example can be found in `preface.tex`. The table of contents is generated automatically with the `\tableofcontents` command. Chapters are included after `\mainmatter` and appendices after `\appendix`. For example, `\input{chapter-1}` includes `chapter-1.tex`, which contains this introduction.

1.2. Bibliography

The bibliography, finally, is generated automatically with

```
\printbibliography[heading=bibintoc]
```

from `bib/report.bib`. The bibliography style is specified in `htwg-report.cls`. As an example, we cite the paper by Nobel Prize winner Andre Geim and his pet hamster [GT01]. If you need to use a different style, change

```
%% BIB
\RequirePackage[
```

²Note that it is not necessary to specify the file extension.

```

        backend=biber,
        style=alphabetic,
        sorting=ynt
    ]{biblatex}
\DefineBibliographyStrings{english}{%
    bibliography = {References},
}

```

As compiler, use `biber` to compile and generate the bibliography.

1.3. Cover and Title Page

This template will automatically generate a cover page if you issue the `\makecover` command. However, before generating the cover, you need to provide the information to put on it. This can be done with the following commands:

```

%% 'reporttype' add background elements to the cover / front page
%% possible values are:
%% bachelor      --> B S C
%% master        --> M S C
%% other         --> none
\reporttype{bachelor}

\reporttypetext{Bachelor Thesis}

```

1.4. Chapters

Each chapter has its own file. For example, the $\text{\LaTeX}$ source of this chapter can be found in `chapter-1.tex`. A chapter starts with the command

```
\chapter{Chapter title}
```

This starts a new page, prints the chapter number and title and adds a link in the table of contents. If the title is very long, it may be desirable to use a shorter version in the page headers and the table of contents. This can be achieved by specifying the short title in brackets:

```
\chapter[Short title]{Very long title with many words which could
not possibly fit on one line}
```

Unnumbered chapters, such as the preface, can be created with `\chapter*{Chapter title}`. Such a chapter will not show up in the table of contents or in the page header. To create a table of contents entry anyway, add

```
\addcontentsline{toc}{chapter}{Chapter title}
```

after the `\chapter` command. To print the chapter title in the page header, add

```
\setheader{Chapter title}
```

Chapters are subdivided into sections, subsections, subsubsections, and, optionally, paragraphs and subparagraphs. All can have a title, but only sections and subsections are numbered. As with chapters, the numbering can be turned off by using `\section*{...}` instead of `\section{...}`, and similarly for the subsection.

1.5. `\section{...}`

1.5.1. `\subsection{...}`

`\subsubsection{...}`

`\paragraph{...}` Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

1.6. Fonts and Colors

If you want to use the HTWG house style font Swiss 721 it is necessary to put the font as .ttf file under fonts. As fallback font Arial is used. For more informations to the HTWG house style font see <https://www.htwg-konstanz.de/hochschule/einrichtungen/stabsstelle-kommunikation/corporate-design-logo/>.

2

Grundlagen und Technologien

In diesem Kapitel werden die Grundlagen und die verwendeten Technologien erläutert, die für die Konzeption und Umsetzung der Kompiliereranbindung für CodeTask von Bedeutung sind. Zunächst wird das Konzept von Online-Judge-Systemen im Allgemeinen betrachtet, bevor auf die spezifischen Isolationsmechanismen und Kommunikationsprotokolle eingegangen wird.

2.1. Online-Judge-Systeme

Ein Online-Judge-System ist eine Plattform, welche Programmieraufgaben zur Verfügung stellt die von Nutzern gelöst werden können. Dazu wird der Code gegen vordefinierte Testfälle geprüft. Durch solch ein System können sich Studenten, Schüler aber auch Berufstätige in unterschiedlichen Programmiersprachen zu allen möchtlichen Themen fortbilden ohne sämtliche Systeme selbst aufsetzen zu müssen. Dazu sorgt solch ein System für eine einheitliche, objektive Bewertung - gleiche Eingabe, gleiche Ausgabe, es gibt keine Sympathie, die bevorzugen oder benachteiligen kann. Zusätzlich bekommt man ein sofortiges Ergebnis. Ein Computer kann die Aufgaben schneller bewerten als jeder Mensch. Ein Online-Judge kann in unterschiedlichsten Möglichkeiten genutzt werden. Es kann regulär zum lernen/fortbilden genutzt werden, in Wettbewerben in denen man gemessen werden kann oder als Portfolio zum offenlegen aktueller Fähigkeiten.

2.1.1. Architektur eines Online-Judges

Ein jedes Online-Judge-System besteht aus einer handvoll Bausteinen die für einen reibungslosen Betrieb von nöten sind.

User-Interface

In irgendeiner Form muss der Nutzer seinen Code abgeben können. Hier kommt das Frontend in der Regeln als eine Website ins Spiel. In einem Textfeld wird der Code eingegeben und zum prüfen an das Backend geschickt.

Aufgaben-Management

Damit das System weiß, welche Aufgaben und Nutzer es gibt, wird eine Datenbank benötigt. In dieser werden sämtliche notwendigen Informationen permanent abgespeichert. Darunter fallen auch Ergebnisse von Nutzern oder Statistiken. Damit die Aufgaben im Frontend angezeigt werden können, müssen die Informationen sowohl erstmals in der Datenbank gespeichert, als auch kontinuierlich gelesen werden. Dafür sorgt ein Management Service. Dieser Service verbindet das Frontend mit der Datenbank und ermöglicht die notwenige Kommunikation.

Judging-Engine / Compiler

Jedes dieser Systeme muss den Programmcode des Nutzers ausführen um zu verifizieren, dass dieser das korrekte Ergebnis liefert. Das passiert mit einer Judging-Engine. Diese nimmt die Anfragen mit dem Code entgegen und führt den Code aus. Das ganze passiert dabei in der Regel asynchron, damit das System auch bei vielen Anfragen nicht zusammenbricht. Sobald die Judging-Engine den Code ausgeführt hat und ein Ergebnis erstellt hat, wird dieses zurück geschickt.

Sandbox

Der kritischste Teil eines solchen Systems liegt in der sogenannten dynamischen Code-Ausführung. Dabei ist gemeint, das in einem Programmcode ein Compiler eingebaut ist, welcher während der Laufzeit des Programmcodes selbst neuen, zur Laufzeit erstellten, Programmcode erhält, kompiliert und ausführt. Dynamische Code-Ausführung ist dabei sehr häufig ein großes Risiko, da der Code vollen Zugriff auf das System hat, in dem er läuft. Das kann bedeuten, das der Code gefährliche Befehle wie zum Beispiel `sys.exit()` ausführen kann und damit das gesamte Programm beenden oder sogar Dateien auslesen kann. Um diesem Problem zu entweichen kann man so ein System in einer Sandbox aufbauen. Dabei ist eine Sandbox ein abgesichertes System das nur eingeschränkt arbeiten kann. Bei richtiger Anwendung einer Sandbox können so sämtliche Sicherheitslücken umgangen werden und dynamische Code-Ausführung wäre sicher.

2.1.2. Der Prozessablauf

Einreichung

Die Reise startet im Frontend. Der Nutzer wählt eine Aufgabe aus, die er lösen möchte. Darauf hin wird diese Aufgabe mit Tests und eventuellem vorgegebenen Code aus der Datenbank geladen und für den Nutzer angezeigt. In den gängigen Systemen, auf welche ich im Absatz [] genauer eingehe, wird der Code in ein Text-Widget geschrieben (genauer zu solchen Text-Widgets im Absatz []). Sobald der Nutzer fertig programmiert hat, kann er seine Lösung abschicken. Durch das Abschicken werden alle Relevanten Daten an das Backend geschickt. Im Backend werden weitere benötigte Daten aus der Datenbank geladen und zusammen an die Judging-Engine gegeben.

Validierung

Ist der Code überhaupt vollständig beziehungsweise Syntaktisch korrekt? Das wird je nach Programmiersprache unterschiedlich Entschieden. In Sprachen wie C, Java oder Scala ist hier ein vorläufiges überprüfen des Codes hilfreich. Durch Untersuchen des Codes auf bestimmte Schlüsselwörter, kann so frühzeitig erkannt werden ob der Code böseartig ist.

Kompilierung

Ist die Validierung erfolgreich, kann der Code kompiliert werden. Dafür kann ein vorhandener Kompiler genutzt oder eigener geschrieben werden. Damit der Kompiler den Code aber in Maschinensprache übersetzen kann müssen von dem Kompiler noch einige Prüfungen stattfinden. Darunter prüfen ob die Vokabeln korrekt sind, also ob die Schlüsselwörter wie val oder if richtig geschrieben wurden. Daraufhin wird die Syntax geprüft, also ob der Satzbau korrekt ist, zum Beispiel ob eine Klammer oder ein Semikolon vergessen wurde. Und zum Schluss findet die semantische Prüfung statt, hier wird überprüft, ob der Code Sinn ergibt. Einen Text durch eine Zahl zu teilen wäre dabei ein Szenario bei dem diese Prüfung anschlagen würde. Ist bis hierhin alles fehlerfrei, übersetzt der Kompiler den Code in eine ausführbare Maschinensprache.

Ausführung

Der Code ist übersetzt und kann ausgeführt werden. Der Code liefert am Ende ein beliebiges Ergebnis. Dieses muss jedoch überprüft werden, damit beurteilt werden kann, das der Code auch die gestellte Aufgabe löst.

Bewertung

Um den Code bewerten zu können brauche wir Testfälle, die mit dem Code zusammen kompiliert wurden. Diese vergleichen das erzeugte Ergebniss des Codes mit der erwarteten Lösung.

Rückmeldung

Stimmt das Ergebnis mit dem Test überein, so wird dem Nutzer dies mitgeteilt. Ist das Ergebnis falsch oder der Code zu komplex, sodass das Programm zu lange ausgeführt werden muss, wird auch das dem Nutzer beigebracht.

Für Scala bietet sich direkt der Scala-Kompilierer Dotty [EPF25] an.

2.1.3. Vergleich bestehender Systeme

HackerRank und LeetCode

LeetCode [Lee] und HackerRank [Hac] sind aktuell die größten Online-Judge-Systeme. Beide Unternehmen bieten auf Ihren Websites mehrere hundert unterschiedliche Programmieraufgaben in unterschiedlichen Programmiersprachen an. Ihr Ziel ist es, Studenten, Auszubildende, Quereinsteiger, aber auch Angestellte Entwickler weiter zu bilden. LeetCode's Aufgaben sind dabei sehr stark an die gestellten Aufgaben von riesen Tech Firmen wie Google, Meta, Amazon und co. angelehnt. Damit ist LeetCode ideal geeignet, um sich auf Codeinterviews vorzubereiten. LeetCode fokussiert sich dabei stark auf algorithmische Aufgaben, wie Data Structures oder Algorithms. HackerRank auf der anderen Seite wird zwar genau so für Interviews direkt genutzt, bietet aber häufiger Coding Challenges und Wettbewerbe an, in denen sich die Nutzer untereinander vergleichen, messen und motivieren können. Dazu ist das Angebot an Themen auch breiter. Datenbanken, KI, Sicherheit und Linux und weitere Aufgabenbereiche werden hier angeboten. Beide Unternehmen können tausende gleichzeitige Nutzer verarbeiten. Das erreichen sie durch Asynchrone Verarbeitung von Anfragen, bei denen eine Anfrage erstmal in eine Warteschlange abgelegt wird. Der Nutzer bekommt sofort eine Benachrichtigung, dass die Anfrage im System eingegangen ist, die tatsächliche Verarbeitung der Daten passiert aber erst danach. Damit die Warteschlange schnell abgearbeitet werden kann, auch unter Last, kommt Horizontales Scaling in Einsatz. Dabei können auf Abruf weitere Dienste hochgefahren werden, welche dabei helfen, die Anfragen zu verarbeiten. Die Anfragen werden zudem in isolierte Container verlagert. Große Systeme wie das von LeetCode oder HackerRank stellen dafür tausende wartende Container bereit, die darauf warten, den Code des Nutzers auszuführen. Auf Container gehe ich dabei im Detail in Abschnitt 2.2.1 ein. Als letzte Skalierung werden Lastverteiler (Load Balancer) genutzt. LeetCode und HackerRank haben auf der ganzen Welt Server verteilt. Anfragen aus einem bestimmten Land werden dabei automatisch an den nächstgelegenen Server geleitet, sodass Latenzen möglichst klein gehalten werden können. Load Balancer merken allerdings, wenn der Server überlastet ist und leiten die Anfrage in so einem Fall automatisch an einen Server mit weniger Last weiter.

Sphere Online Judge (SPOJ)

SPOJ [Spo], aus den frühen 2000er Jahren, ist eines der ersten Judging Systemen. SPOJ hat es jedem ermöglicht eigene Aufgaben zu erstellen und online hochzuladen. So konnte Code in über 40 Programmiersprachen, darunter auch exoten wie Intercal oder Brainfuck, getestet werden. Allerdings musste für jede Sprache ein Compiler oder Interpreter eingebaut sein, was diese extreme Vielfalt extrem wartungsintensiv macht. SPOJ hat damit aber den Grundstein für das Grundprinzip Code → Test → Ergebnis gelegt, welches 20 Jahre später noch genau so angewandt wird. Im Vergleich zu LeetCode liegt bei SPOJ der Fokus auf reiner Informatik. Hier geht es darum, arithmetische Probleme zu lösen und weniger darauf sich für das nächste Code-Interview vorzubereiten. SPOJ setzt auf ein sehr striktes Bewertungssystem, das auch auf Faktoren wie Zeit und Speicherlimits achtet um die Effizienz des Codes zu prüfen.

CodeTask

In unserem Hochschul spezifischen Projekt CodeTask, um das es in dieser Arbeit geht, liegt der Fokus vorerst auf der Lehre der Studenten in der Programmiersprache Scala. CodeTask soll sowohl in Vorlesungen als auch in Prüfungen von den Dozenten und Studenten genutzt werden. Dafür können Lehraufgaben direkt vom Dozent oder Tutoren selbst erstellt und bereitgestellt werden. Das Ziel ist nicht, dass der Student ein Wahr oder Falsch bekommt, sondern klare Antwort die für einen idealen Lernfortschritt sorgt. Aktuell ist CodeTask noch in der Entwicklung und primär auf die Nutzung innerhalb des Hochschulnetzwerks der HTWG-Konstanz begrenzt. Damit Datenschutz und viel wichtiger, die Kontrolle über studentische Daten garantiert werden können läuft die Applikation aktuell auf Hochschul eigenen Servern und nicht wie bei LeetCode auf riesigen externen Cloud-Infrastrukturen. Ein Nachteil liegt dabei darin, dass das System effizient auf Hochschul-Servern funktionieren muss. Dafür ist es allerdings einfacher auf Services die LDAP (Hochschul-Login) und weitere Tools der Hochschule zuzugreifen. CodeTask ist eine Eigenentwicklung der HTWG-Konstanz, welche zum aktuell Stand vorwiegend auf Scala-Übungen ausgelegt ist. Die im Rahmen meiner Bachelorarbeit entwickelte Compileranbindung bildet dabei die entscheidende technische Grundlage, um die Programmiersprache Scala in das System zu integrieren und gleichzeitig eine universelle Schnittstelle für die zukünftige Einbindung weiterer Programmiersprachen zu ermöglichen.

Vergleichstabelle und Analyse

Zur Verdeutlichung der Unterschiede zwischen den vorgestellten Systemen und der Einordnung von CodeTask dient die folgende Tabelle 2.1. Diese stellt die kommerziellen Plattformen, historische Pioniere und die spezialisierte Hochschullösung anhand ausgewählter Kriterien gegenüber.

Tabelle 2.1: Vergleich der Online-Judge-Plattformen

Kriterium	HackerRank / LeetCode	SPOJ	CodeTask
Zielgruppe	Globales Recruiting / Wettbewerbe	Competitive programming	Pro-Hochschullehre (HTWG)
Infrastruktur	Cloud-basiert (global)	Server-basiert (zentral)	Lokal (Hochschul-Server)
Sprachvielfalt	Sehr hoch (> 30)	Extrem hoch (> 40)	Fokus auf Lehre (Scala)
Isolation	Cloud-Container	Klassische Jails	Docker-Container / Kubernetes Cluster
Open Source	Nein (Kommerziell)	Nein	Ja (Eigenentwicklung)

Wie in Tabelle 2.1 zu erkennen ist, belegt CodeTask eine etwas andere Rolle. Während HackerRank und LeetCode stark auf Skalierbarkeit für Millionen von Nutzern ausgelegt sind liegt der Fokus bei CodeTask auf der direkten Einbindung des Lehrplans und der Kontrolle über die studentischen Daten. Die in dieser Arbeit entwickelte Anbindung nutzt dabei mit Docker und Kubernetes Clustern eine Technologie, die auch bei den großen Unternehmen als Standard für die Isolation gilt, passt diese jedoch auf die spezifischen Ressourcen der HTWG an.

2.2. Containerisierung mit Docker

2.2.1. Das Docker-Prinzip (Images und Container)

Um die Ausführung von Nutzercode für das System sicher zu machen, wird Isolation benötigt. Es braucht eine Möglichkeit Code auf einem Server ausführen zu können, ohne dabei das gesamte System durch fehlerhaften oder bösartigen Code zu gefährden. Hier kommt in unserem Fall Docker [Doc26] in Einsatz.

Docker-Images

Docker-Images sind der Grundstein von Docker. Sie können mithilfe einer sogenannten Dockerfile erstellt werden. 2.1 zeigt hier solch eine Dockerfile. Als erstes wird ein passendes Image ausgewählt. Ein Image ist in diesem Fall ein eigenes kleines Betriebssystem, welches später in den erstellbaren Containern ausgeführt wird und die Basis für Codeausführung oder andere Aufgaben bildet. Nach dem das Image ausgewählt ist, wird ein Ordner erstellt, der das Quellverzeichnis darstellt. Daraufhin werden alle notwendigen Dateien in dieses Quellverzeichnis kopiert und zum Schluss kann ein Befehl ausgeführt werden, welcher z.B. den Code aus dem Quellverzeichnis kompiliert und ausführt. In einem Docker-Image befinden sich dann all diese Informationen. Die Dockerfile aus 2.1 stellt dabei eine leicht veränderte Version der Dockerfile da, mit welcher das Image für die Entwicklungsumgebung der Check-API von CodeTask erstellt wird. Das Produktions-Image ist dabei etwas komplexer und optimierter für eine effizientere Nutzung auf Server.

Listing 2.1: Beispiel eines Dockerfiles für den Compiler-Service

```
# Als Basis wird ein Scala/JDK Image genutzt
FROM sbtscala/scala-sbt:eclipse-temurin-21.0.7_6_1.11.2_3.7.1

# Arbeitsverzeichnis im Container erstellen
WORKDIR /app

# Notwendige Dateien fuer den Compiler-Service kopieren
COPY check-api /app

# Befehl zum Starten des persistenten Compiler-Dienstes
CMD ["sbt", "run"]
```

Docker-Container

Mit Docker-Images können beliebig viele Docker-Container erstellt werden. Dafür kommt der Befehl `docker run [Docker-Image Name]` in Einsatz. Wird dieser Befehl ausgeführt, nutzt Docker die Informationen aus dem angegebenen Docker-Image um einen Docker-Container zu erstellen. Bei der Erstellung des Containers wird eine neue Schicht erstellt, die auf die Daten des Docker-Images lese-Zugriff hat und so die Informationen nutzen kann ohne das Image zu verändern. In dem Container wird in unserem Beispiel 2.1 daraufhin die Befehlskette `sbt run` ausgeführt. Damit wird der Code aus dem Quellverzeichnis des Docker-Image kompiliert und ausgeführt. Somit wird unser Programmcode in einem eigenen Docker-Container ausgeführt. Der Docker-Container existiert dabei solange bis der Code beendet wird oder der Docker-Container gelöscht wird.

2.2.2. Isolation und Sicherheit (Sandboxing)

Ein Docker-Container kann wie ein eigenes System angesehen werden (Sandbox). Sofern nicht explizit eine Kommunikationsmöglichkeit nach außen angegeben wird, kann nichts, das innerhalb des Docker-Containers passiert auf das darunter liegende Host-System zugreifen. Diese Technologie sorgt dafür, dass wir die Check-API abgesichert ausführen können ohne das restliche System zu gefährden. Da die Check-API zur Laufzeit Code ausführen muss, kann es passieren, dass dieser Code schadhaft ist. So könnte der auszuführende Code Befehle beinhalten, welche Betriebssystemfunktionen ausführen die über die Anwendung hinausgehen. Dadurch könnten externe Prozesse oder Änderungen von Systemeinstellungen vorgenommen werden. Das wird verhindert, wenn wir den Code in einem Docker-Container ausführen. Hier kann der Code zwar immer noch Schaden anrichten, wie, den Docker-Container selbst zu beenden, in dem die laufende Check-API beendet wird.

2.3. Orchestrierung mit Kubernetes

Docker-Container können, wie auch in unserem Fall, durch die Nutzung von Kubernetes (K8s) [The26] noch weiter verbessert werden.

2.3.1. Grundlagen der Container Verwaltung

2.3.2. Hochverfügbarkeit und Self-Healing

3

Anforderungen

4

Konzeptarchitektur

5

Implementierung

6

Evaluation

7

Fazit

Literatur

- [GT01] A. K. Geim und H. A. M. S. ter Tisha. “Detection of earth rotation with a diamagnetically levitating gyroscope”. In: *Physica B: Condensed Matter* 294–295 (2001), S. 736–739. DOI: [10.1016/S0921-4526\(00\)00753-5](https://doi.org/10.1016/S0921-4526(00)00753-5).
- [EPF25] EPFL. *The Scala 3 compiler, also known as Dotty*. Version 3.6.3. 2025. URL: <https://github.com/scala/scala3> (besucht am 10.06.2026).
- [Doc26] Docker Inc. *Docker*. 2026. URL: <https://www.docker.com> (besucht am 16.06.2026).
- [The26] The Kubernetes Authors. *Kubernetes*. 2026. URL: <https://kubernetes.io/de/> (besucht am 16.06.2026).
- [Hac] HackerRank. *HackerRank*. URL: <https://www.hackerrank.com> (besucht am 14.06.2026).
- [Lee] LeetCode. *LeetCode*. URL: <https://leetcode.com> (besucht am 14.06.2026).
- [Spo] Spoj. *Spoj.com*. URL: <https://www.spoj.com> (besucht am 14.06.2026).